

Forme A Structured Visual Website Builder with Portable HTML and CSS Export

Shehroz Riaz
Web Engineering Project

Abstract—This paper presents Forme, a browser-based visual editor for constructing single-page websites from predefined sections. The system represents a page as a validated JSON document and uses one renderer for both an isolated design preview and portable HTML/CSS export. A React interface provides section insertion, reordering, content and style controls, history navigation, and viewport presets. An Express service supports authentication and private project storage through interchangeable local-file and MongoDB adapters. Evaluation of the initial prototype consists of six automated tests and a production build, supplemented by limited browser observations. All automated tests pass, including checks for project ownership, persistence, validation, and output escaping. These results establish correctness for the exercised paths rather than usability, scalability, or deployment security. The paper explains the architecture, implementation choices, observed results, and remaining work, including MongoDB integration, complete browser testing, and production hardening.

Index Terms—visual website builder, web engineering, React, structured page model, responsive design, code generation

I. INTRODUCTION

Constructing even a small website requires coordination between document structure, visual styling, responsive behavior, and deployment. A visual editor can expose these decisions through direct controls while preserving an understandable relationship to the underlying HTML and CSS. The engineering problem is to maintain a coherent document as users insert, move, restyle, and export its parts.

Forme addresses this problem through section-level composition. A user starts with a template or blank page, adds semantic sections, edits their fields, and inspects the result at several viewport widths. The exported artifact is an ordinary website containing HTML and CSS. It does not require the React editor or a running account service.

The project investigates whether a constrained page model can support visual authoring, account-based persistence, and predictable code generation within the scope of an undergraduate web engineering implementation. Its contribution is an implemented and inspectable prototype, rather than a claim of a novel visual-programming technique or measured superiority over existing builders.

The current scope deliberately limits editing to a single page and eight section types. Free positioning, arbitrary scripts, collaborative editing, and two-way source editing are excluded. These boundaries reduce the number of states that the renderer and validation layer must handle.

II. BACKGROUND AND DESIGN OBJECTIVES

React encourages decomposing an interface into components and identifying the state required to represent user interaction [1]. Forme applies this approach to the library, layer list, canvas, property inspector, and account dialogs. Its implementation keeps the page document separate from selection, viewport mode, and history controls.

Structured project persistence is established practice in visual editors. GrapesJS documents JSON project data as the appropriate persistence representation and warns that exported HTML/CSS may omit editor information [2]. Forme adopts a similar separation: JSON preserves editable state, while HTML/CSS is the delivery format. It does not embed or extend GrapesJS.

Three design objectives guide the implementation. First, editing and export should derive from the same document and rendering rules. Second, the generated site should remain usable without a builder-specific runtime. Third, persisted account projects should be accessible only to their owners. These objectives are implemented as architectural boundaries and checked where feasible with automated tests.

TABLE I. INITIAL FEATURE SCOPE

Area	Implemented scope
Composition	Eight section types; insertion, reordering, duplication, deletion
Appearance	Section colors, spacing, alignment; page font, accent, radius
Inspection	Design and code views; three viewport presets
Persistence	Browser draft; authenticated project CRUD
Export	ZIP with HTML, CSS, and instructions

The eight section types are navigation, hero, feature grid, text, image, testimonial, call to action, and footer. Three starter templates provide studio, portfolio, and product layouts. A blank option supports composition from an empty document. The editor supplies useful defaults while keeping its output accessible for later manual modification.

This work reports a project implementation in an IEEE-style layout. It does not imply IEEE publication, peer review, or an empirical user study.

III. SYSTEM ARCHITECTURE

A. Separation of authoring and delivery

The browser hosts a React editor and a sandboxed iframe. Editing actions update the page document. The shared renderer transforms that document into HTML and CSS. The preview inserts the generated CSS into the iframe document and, in design mode, adds a small selection script. Export writes the HTML and CSS as separate files without this editor script.

The account and project interface calls an Express REST API on the same origin. The API handles credentials, sessions, validation, and owner checks, then delegates storage through a small adapter interface. Development uses a local JSON file. The optional MongoDB adapter provides the same get, find, put, remove, and close operations.

TABLE II. ARCHITECTURAL RESPONSIBILITIES

Component	Responsibility
React editor	Capture changes and manage interaction state
Page model	Represent typed sections, content, and styles
Shared renderer	Generate preview and export markup
Sandboxed iframe	Display generated page and report selection
Express service	Authenticate and validate private project operations
Storage adapter	Persist users, sessions, and projects

B. Document and record design

A page stores a schema version, a theme object, and an ordered sections array. Each section has a unique identifier, an allowed type, a content object, and style properties. Theme values include font family, accent color, and corner radius. Section styles include foreground and background colors, vertical padding, and text alignment.

A project adds its identifier, owner identifier, name, and creation/update timestamps. User records store normalized email addresses and salted password hashes. Session records contain a hash of an opaque token, the user identifier, and an expiry value. These records support authorization independently of the page content.

IV. EDITOR AND RENDERING IMPLEMENTATION

A. State updates and history

A selected section determines which fields appear in the inspector. Updates produce a replacement page document rather than mutating the prior state in place. The previous document is placed in a bounded undo history; a new change clears redo history. Section movement operates on a cloned array, preserving identifiers and content. The history retains up to 60 page snapshots.

The library inserts a section after the selection or at the end when no selection exists. The layer list supports drag reordering and explicit movement buttons. Duplication creates a new identifier so that the copy can be selected and edited independently. Browser localStorage retains one draft after a short debounce; this draft is separate from account project saving.

B. Validation and generation

Validation checks schema version, permitted section types, unique identifiers, required content fields, and bounded style values. A document may contain at most 60 sections. Text fields are limited to 5,000 characters, and the service limits JSON request bodies to 512 KB. Invalid documents are rejected before account storage and before rendering.

The renderer dispatches each section to a semantic markup template and emits shared responsive rules plus section-specific CSS. It escapes text for HTML output, restricts URL protocols, and permits only constrained style values. Generated HTML references styles.css. The ZIP exporter adds both files and a short usage guide.

C. Responsive preview and portability

The canvas renders at widths of 1200, 768, or 375 CSS pixels for desktop, tablet, and mobile presets. A visual scale fits that viewport into the available workspace without changing its internal layout width. Exported CSS uses flexible sizing and a 640-pixel breakpoint to stack the hero, feature grid, and text layout.

The live code view is a read-only projection of the current document. Two-way parsing is excluded because arbitrary source edits cannot always be mapped back into the constrained section model without loss. Remote images remain URL references in exports, so they require network access and are not self-contained assets.

V. PERSISTENCE AND SECURITY CONTROLS

A. Authentication and project ownership

The service normalizes email addresses and derives password hashes with Node.js `scrypt` using a random salt. Login compares equal-length derived values with `timingSafeEqual`. Successful authentication issues a cryptographically random token. Its SHA-256 digest is stored on the server, while the token is carried in an HTTP-only cookie with `SameSite=Lax` and a seven-day lifetime [7].

The Secure cookie flag is enabled in production, requiring HTTPS. Every project route requires an authenticated user. Routes addressing an individual project also check the owner identifier; a missing or non-owned project returns the same not-found response. This prevents the interface from being the only enforcement point.

The API rejects conflicting Origin headers and cross-site Fetch Metadata on mutation requests, and requires JSON for POST/PUT operations. Authentication routes have a request-rate limit. These measures reduce specific attack opportunities, but they do not establish comprehensive security compliance or replace a deployment review.

B. Preview boundary and output safety

The `iframe` permits its selection script but omits same-origin privileges. MDN describes sandbox restrictions and cross-window messaging as separate browser mechanisms [3], [4]. The parent checks the message source against the current `iframe` window and verifies that a reported section identifier exists. The preview accepts highlight messages only from its parent. Messages carry selection identifiers, not credentials.

Escaping and URL checks remain necessary even inside a sandbox because the export runs as a normal page. The generator neutralizes unsupported schemes and rejects invalid style strings. The automated renderer tests cover representative script injection and URL cases; they are not an exhaustive fuzzing or penetration test.

C. Production limitations

The implementation currently uses Node.js default `scrypt` work parameters. These should be reviewed and upgraded against OWASP guidance before public authentication deployment [5], [6]. Email verification, password recovery, distributed rate limiting, and a full content-security policy remain unimplemented. Browser drafts are shared within a browser profile and must not be described as private account storage.

VI. EVALUATION METHOD AND RESULTS

A. Reproducible checks

Evaluation uses the repository [8] and its Node test runner and Vite production build. The renderer tests exercise every template and section type, standalone output, representative unsafe inputs, schema rejection, and immutable reordering. A service test creates two accounts and exercises signup, login, project creation, listing, updates, persistence, cross-account access rejection, deletion, logout, and invalid-origin handling.

The service test passes request streams through the actual Express middleware stack without binding a TCP port. This permits deterministic checks in a restricted environment, but excludes network behavior and reverse-proxy configuration. Persistence is checked by reopening the local store and reading an updated project.

TABLE III. OBSERVED VALIDATION RESULTS

Check	Observed result
Template and block rendering	Pass
Standalone export structure	Pass
Escaping and URL restrictions	Pass
Invalid schema rejection	Pass
Immutable section reordering	Pass
Account and project lifecycle	Pass
Vite production build	Pass
MongoDB integration	Not tested

All six automated tests passed in the recorded run. The production build also completed. Its reported JavaScript bundle was 334.53 kB, or 104.85 kB with gzip; CSS was 21.72 kB, or 5.64 kB with gzip. These are build-artifact sizes, not download measurements or page-speed scores. They describe the editor bundle, not every exported website.

B. Browser observations

The existing browser session displayed the Studio North project. Switching to Code showed the generated HTML, and switching to the mobile canvas changed the viewport preset to 375 pixels. These observations confirm specific interface transitions only. Complete editing, account saving, export-download, and cross-browser workflows require further live testing.

No participant study, comparative benchmark, load test, or statistical analysis was performed. A passing test count must therefore not be interpreted as complete coverage, proven usability, or production readiness.

VII. DISCUSSION AND FUTURE WORK

A. Implications of a constrained model

Using one page representation makes the relationship between an edit, the preview, and exported source explicit. The shared renderer avoids maintaining separate template systems for authoring and delivery. Nevertheless, shared code alone cannot prove visual equivalence: the preview adds selection behavior, the editor scales its iframe, and external resources may load differently outside it.

Section-level composition trades flexibility for a smaller implementation surface. It works well for the provided landing-page patterns but cannot represent arbitrary nested layouts. A future component model could support deeper composition, provided it retains clear constraints and a migration strategy for saved documents.

Some current navigation behavior is intentionally limited. Starter links target fixed work, about, and contact anchors. Repeated copies of those sections can introduce duplicate named anchors, and footer link labels are currently display text. A dedicated anchor and link editor should precede claims of unrestricted navigation support.

B. Storage and deployment

The JSON adapter serializes writes through a process-local queue and replaces the file through a temporary path. It is appropriate for a single-process demonstration, not a distributed deployment. The MongoDB adapter and indexes are implemented, but the driver remains an optional setup step and the database path has not been integration-tested.

The next validation stage should run the same account and ownership cases against MongoDB, then test the application behind an HTTPS reverse proxy. Session expiry, concurrent updates, malformed requests, database failures, and recovery behavior deserve dedicated checks. The project currently lacks revision conflict detection for simultaneous saves.

C. Planned evaluation

A future usability study can ask participants to create and export a small site from a fixed brief. Appropriate measures include task completion, time to completion, recovery from an accidental deletion, and the number of assistance requests. A paired comparison could evaluate Forme against manual HTML/CSS for the same brief, with participant experience recorded and order effects controlled.

Engineering extensions should begin with image upload, unique anchor management, keyboard interaction testing, and regression screenshots. Version history and multi-page export can follow once the initial save and export workflows have been evaluated. These are proposed directions rather than implemented results.

VIII. CONCLUSION

Forme implements a visual authoring workflow around a constrained JSON page model. React handles editing, a shared renderer generates preview and exported source, and an Express service applies authentication and ownership checks to persisted projects. The initial prototype includes eight section types, three templates, responsive canvas presets, local draft recovery, and standalone HTML/CSS export.

Six automated tests and a successful production build provide evidence for the exercised implementation paths. Limited browser observations additionally verify code-view and mobile-preview transitions. The current evidence does not establish complete visual correctness, usability, scalability, or production security. Further work should validate the MongoDB path, complete browser workflows, and strengthen deployment controls before expanding the feature set.

REFERENCES

- [1] React, “Thinking in React.” Accessed Oct. 9, 2026. [Online]. Available: <https://react.dev/learn/thinking-in-react>
- [2] GrapesJS, “Storage Manager.” Accessed Oct. 9, 2026. [Online]. Available: <https://grapesjs.com/docs/modules/Storage.html>
- [3] MDN Web Docs, “The iframe element.” Accessed Oct. 9, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/iframe>
- [4] MDN Web Docs, “Window.postMessage method.” Accessed Oct. 9, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>
- [5] OWASP Foundation, “Password Storage Cheat Sheet.” Accessed Oct. 9, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [6] Node.js, “Crypto.” Accessed Oct. 9, 2026. [Online]. Available: <https://nodejs.org/api/crypto.html>
- [7] OWASP Foundation, “Session Management Cheat Sheet.” Accessed Oct. 9, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [8] S. Riaz, “Forme website builder,” source code, project documentation, and local validation records, version 0.1.0, Oct. 2026.